COVID-19 Data Analysis in Black and White with Python Pandas

The COVID-19 pandemic has been the defining feature of the last 18 months the related data has been the focus of much research. Many organisations including Google, and the UK Government, have made this data publicly available, so that it can be analysed by academics, students and other interested parties. Such analysis can be achieved using a variety of tools from MatLab, through R to Python and its Data Analytics stack that includes the Pandas data analytics library and Matplotlib graphing library.

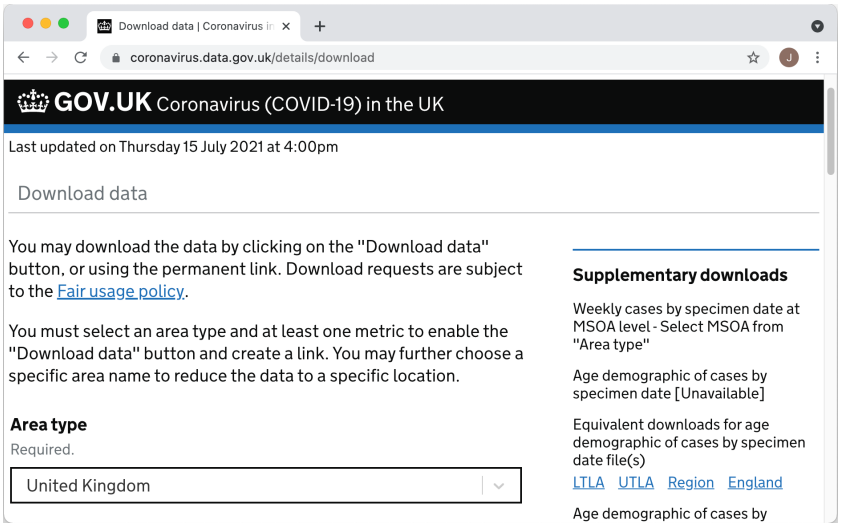
In this blog we will look at two data sets that can be obtained free from public sources and how the Python Pandas library can be used to perform some simple data analytics.

The Data

We shall be using two datasets to explore data around the COVID pandemic through the python Pandas library. These data sets are described below.

The UK Government COVID Dataset

The first dataset is provided by the UK Government as part of its Coronavirus (COVID-19) data provision. The web site is illustrated below:



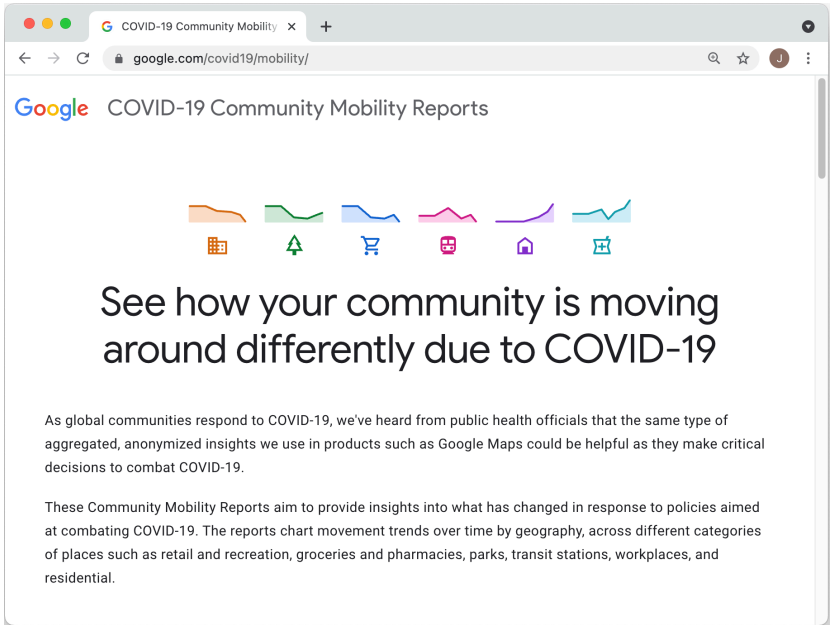
On this site the user can select from various criteria to identify the region, the metrics, the date range of interest and the data format they require. For this blog we selected the UK as the area, the latest data available for 2020, the CSV data format (as that is easy to work with) and a set of metrics to download. The metrics selected were daily hospital cases, daily new admissions, new COVID cases, as well as the number of people who had their first and second vaccinations, as shown below:

To make it possible to repeat such experiments the web site also provides a permanent link for data request, which is given below:

https://api.coronavirus.data.gov.uk/v2/data?areaType=overview&metric=hospitalCases&metric=newAdmissions&metric=newCasesByPublishDate&metric=newPeopleVaccinatedFirstDoseByPublishDate&metric=newPeopleVaccinatedSecondDoseByPublishDate&format=csv

The Google Mobility Dataset

The second dataset to be used is provided by Google. It is accessible from the Google COVID-19 Community Mobility Reports site illustrated below:



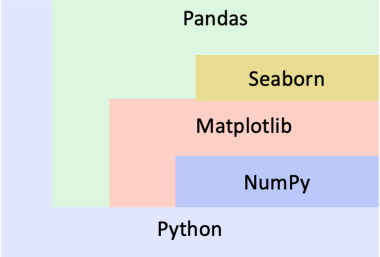
It makes available two files that can be downloaded; the first provides a Global view of mobility while the second provides separate data files for all the regions covered by Google. This second data set is the one which was downloaded for this blog. This download is provided as a ZIP file. When unzipped it contains many different data files for different countries. For this blog the GB data file was selected.

The data is quite wide ranging and includes information on the percentage change of Google users for visits and length of stay at different places such as retail locations, parks, transport hubs etc.

Both datasets will now be processed using the Python Pandas library.

Python Pandas

The Python Pandas library provides facilities that simplify accessing, processing and visualizing data sets. Pandas itself builds on other libraries such the NumPy library. It is also commonly used with graphing libraries such as Matplotlib and Seaborn. The relationship between all of these and Python is illustrated below.



The intention of the above diagram is to illustrate how one library often builds on another but can be used independently or in combination.

Pandas is not provided as a default part of the Python environment; and instead must be installed along with any other 3rd party libraries being used. This can be done using `pip` or `conda` (depending on how you are manging your Python environments). Pip is a tool used to install Python packages and is provided as part of Python. Conda, another package management tool, is part of Anaconda and is a popular choice for data scientists using Python. Both can be used to install Pandas as shown below:

```
pip install pandas
```

```
conda install pandas
```

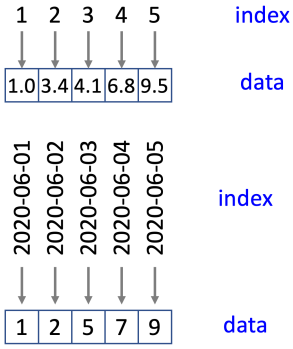
As we will also be using Matplotlib and Seaborn, we will need to install both these 3rd party libraries as well:

```
pip install matplotlib
pip install seaborn
```

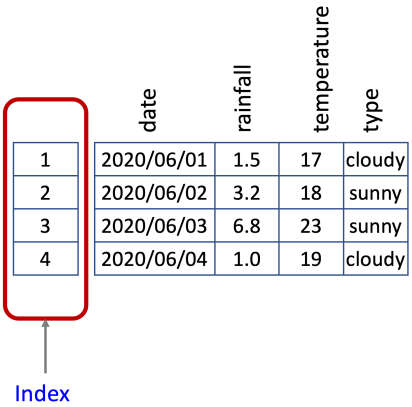
```
conda install matplotlib
conda install seaborn
```

Pandas Series and DataFrames

The key concepts within Pandas are the Series and the DataFrame. A series is a 1-dimensional array like object that holds data of a specific type (such as an integer, a string or a float). Each value is accessible via an index value which can be a numeric integer such as 0 or 1 or a label such as a string or a time stamp. The following diagram illustrates two such Series, one is indexed numerically from Zero and holds a sequence of floats, the second is indexed by a set of timestamps and holds a sequence of integers.



A DataFrame is a tabular structure a bit like a spreadsheet. It has columns and rows. Each column has a type and a label. The columns are represented by Series. Each row has an index. The index could be numeric or some other value such as a sequence of dates. A DataFrame can be visualized as shown below:



We will use DataFrames to load, process and analyse the COVID related data sets.

Loading and Analysing UK COVID dataset

The first thing we will do is load the CSV file, obtained from the UK Government COVID data portal, into a Pandas DataFrame. This is very easy to do with Panadas as it provides a function called `read_csv()` that can be used for this exact purpose. All we need to do is import the Pandas library and then call the function. Note that it is a common convention within the Python words to alias `pandas` to `pd` as a shorthand reference to the library.

The following code loads the dataset into a DataFrame:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()


df1 = pd.read_csv('overview_2021-07-15.csv')
```

The variable `df1` now contains a reference to a Pandas DataFrame. There can be a lot of data held in a DataFrame which can make it hard to visualize, however the functions `head()` and `tail()` can be used to see the first few, and the last few, rows in the DataFrame:

```
print(df1.head().to_string())
print(df1.tail().to_string())
```

The above code obtains the head and the tail of the DataFrame,then converts them into a string format that is suitable for printing. The output from these is given below:

```
    areaCode        areaName  areaType        date  hospitalCases
newAdmissions  newCasesByPublishDate
newPeopleVaccinatedFirstDoseByPublishDate
newPeopleVaccinatedSecondDoseByPublishDate
0  K02000001  United Kingdom  overview  2021-07-15            NaN
NaN                  48553                                     NaN
NaN
1  K02000001  United Kingdom  overview  2021-07-14         3786.0
NaN                  42302                                  60374.0
185661.0
2  K02000001  United Kingdom  overview  2021-07-13         3623.0
NaN                  36660                                  59073.0
158276.0
3  K02000001  United Kingdom  overview  2021-07-12         3415.0
NaN                  34471                                  54296.0
125360.0
4  K02000001  United Kingdom  overview  2021-07-11         3144.0
582.0                  31772                                  42000.0
107620.0
      areaCode        areaName  areaType        date  hospitalCases
newAdmissions  newCasesByPublishDate
newPeopleVaccinatedFirstDoseByPublishDate
newPeopleVaccinatedSecondDoseByPublishDate
527  K02000001  United Kingdom  overview  2020-02-04            NaN
NaN                      0                                     NaN
NaN
528  K02000001  United Kingdom  overview  2020-02-03            NaN
NaN                      0                                     NaN
NaN
529  K02000001  United Kingdom  overview  2020-02-02            NaN
NaN                      0                                     NaN
NaN
530  K02000001  United Kingdom  overview  2020-02-01            NaN
NaN                      0                                     NaN
NaN
531  K02000001  United Kingdom  overview  2020-01-31            NaN
NaN                      2                                     NaN
NaN
```

This is still quite hard to read, not least as there are numerous columns in the data but also because of the presence of a lot of `NaN` values which indicate missing values.

Despite this we can already see a few things about this data, for example, there are *531* rows and that there are several columns which don't provide much in the way of meaningful information for us. For example, we know that this data is about the UK thus `areaCode`, `areaName` and `areaType` which all indicate that this is data about the UK are not

particularly useful and all have the same values. We can therefore choose to drop these columns from the DataFrame:

```
df1.drop(['areaCode', 'areaName', 'areaType'],
         axis='columns',
         inplace=True)
print(df1.head().to_string())
```

The `drop()` method of the DataFrame is given a list of the columns to remove, followed by the `axis` option which indicates its columns we are dropping not rows. The final parameter `inplace` indicates that it should change the DataFrame itself rather than generate a new DataFrame for the result.

Now when we look at the output from the DataFrame we have less columns and less meaningless data:

```
         date  hospitalCases  newAdmissions  newCasesByPublishDate
newPeopleVaccinatedFirstDoseByPublishDate
newPeopleVaccinatedSecondDoseByPublishDate
0  2021-07-15            NaN            NaN                  48553
NaN                                          NaN
1  2021-07-14         3786.0            NaN                  42302
60374.0                                     185661.0
2  2021-07-13         3623.0            NaN                  36660
59073.0                                     158276.0
3  2021-07-12         3415.0            NaN                  34471
54296.0                                     125360.0
4  2021-07-11         3144.0          582.0                  31772
42000.0
```

Of course, we can go further, we can now easily see that there is a date column with individual dates starting from the 15th of July 2021. However, at the moment all that the DataFrame knows about this column is that it contains objects (specifically strings). However, if we tell the DataFrame that this column actually represents dates, or more specifically datetime information, then we can perform date related comparisons on it, such as selecting all rows between a start and end date.

We can tell the DataFrame that the date column should be treated as a set of datetime objects using the Pandas `to_datetime()` function:

```
df1['date'] = pd.to_datetime(df1['date'])
```

This code tells Pandas to convert the `date` column in the `df1` DataFrame to a set of datetime instances and then to overwrite the original column with the new version. Note how columns can be accessed using the square bracket (index) notation.

We can now perform several operations on the DataFrame using the `date` column:

```
# Sort the rows into ascending date order
df1.sort_values(by=["date"],
                ignore_index=True,
```

```
                    inplace=True)
# Want to select 2020-02-15 to 2020-12-31 dates
# Set up a mask to indicate the date election
date_mask = (df1['date'] > '2020-02-14') & (df1['date'] <=
'2020-12-31')
# Select all the rows that meet the mask search criteria
df1 = df1.loc[date_mask]
```

This code snippet does three things:

1. It resorts the rows in the DataFrame into ascending (rather than descending) date order. This is done using the `sort_values()` method available from the DataFrame object. This function takes three parameters:
    a. `by` which is used to indicate the column to use for sorting,
    b. `ignore_index` which indicates that the current index should be ignored and
    c. `inplace` option as used previously.
2. We then want to select just those rows associated with dates between the 14th of Feb 2020 and the 31st of December 2020. This is because we are interested only in the 2020 data and the Google data only covers from 14thof Feb 2020 onwards. To do this we set up a mask. This returns a series of `True` and `False` values for each row in the DataFrame. If a row meets the condition specified in the test, then the value `True` is generated otherwise it is `False`.
3. The DataFrame `loc()` method is then used to select the rows which match the condition in the mask. The `loc()` method will return each row where there is a corresponding `True` in the `date_mask`.

If we now look at the head and tail of the DataFrame we can see:

```
         date  hospitalCases  newAdmissions  newCasesByPublishDate
newPeopleVaccinatedFirstDoseByPublishDate
newPeopleVaccinatedSecondDoseByPublishDate
15 2020-02-15            NaN            NaN                      0
NaN                                         NaN
16 2020-02-16            NaN            NaN                      0
NaN                                         NaN
17 2020-02-17            NaN            NaN                      0
NaN                                         NaN
18 2020-02-18            NaN            NaN                      0
NaN                                         NaN
19 2020-02-19            NaN            NaN                      0
NaN                                         NaN
          date  hospitalCases  newAdmissions  newCasesByPublishDate
newPeopleVaccinatedFirstDoseByPublishDate
newPeopleVaccinatedSecondDoseByPublishDate
331 2020-12-27        22764.0         2870.0                  30501
NaN                                         NaN
332 2020-12-28        24049.0         3133.0                  41385
NaN                                         NaN
333 2020-12-29        25549.0         3249.0                  53135
NaN                                         NaN
334 2020-12-30        26551.0         3288.0                  50023
NaN                                         NaN
335 2020-12-31        26578.0         2915.0                  55892
NaN                                         NaN
```

The above output shows that the *head of the data* starts on the 15th of Feb and the *tail* ends on the 31st of December 2020 and that there are now 335 rows of data.

We can also see that there are still a large number of `NaN` values in the printout. It can be useful to see how many such values there are for each column. We can do this by testing each value in the DataFrame for `NaN` (or nothing or null) and then counting the results for each column. As the DataFrame allows operations to be performed in batch (as efficiently as possible) we can do this using two batch functions `isnull()` and `sum()`:

```
is_null_count = df1.isnull().sum()
print(is_null_count)
```

From this we get:

```
date                                          0
hospitalCases                                41
newAdmissions                                37
newCasesByPublishDate                         0
newPeopleVaccinatedFirstDoseByPublishDate   321
newPeopleVaccinatedSecondDoseByPublishDate  321
```

From this we can see that the last two rows in the above list indicate that the gassociated columns provide virtually no significant information as they contain no significant data (this is probably because the data is from 2020 and prior to the main vaccine roll out in the UK). We can therefore also drop these columns from our DataFrame:

```
df1.drop(['newPeopleVaccinatedFirstDoseByPublishDate',
          'newPeopleVaccinatedSecondDoseByPublishDate'],
         axis='columns',
         inplace=True)
```

Now if we print a random sample of the DataFrame we can see that we have a simpler set of data with mostly meaningful information:

```
# Select a random sample of 10 rows form the DataFrame
print(df1.sample(10).to_string())
```

Some of the data generated by the above statement is given below, note as we used the sample() method the rows are selected randomly and are not in order:

```
      date        hospitalCases  newAdmissions  newCasesByPublishDate
155 2020-07-04         2802.0          202.0                    624
149 2020-06-28         3557.0          208.0                    901
156 2020-07-05         2741.0          196.0                    516
276 2020-11-02        13105.0         1566.0                  18950
331 2020-12-27        22764.0         2870.0                  30501
89  2020-04-29        15521.0         1488.0                   4076
260 2020-10-17         6545.0         1026.0                  16171
112 2020-05-22         9184.0          739.0                   3287
30  2020-03-01            NaN            NaN                     12
```

```
129 2020-06-08          5931.0          412.0                    1205
```

Loading the Google Mobility Dataset

If we now wished to see how movement changed during the 2020 year, we can take the Google Mobility Data set for the UK and perform similar analysis. The following code does exactly this:

```
# Load the google Mobility data for the UK
df2 = pd.read_csv('2020_GB_Region_Mobility_Report.csv',
low_memory=False)

# Drop columns that don;t provide any additional data
df2.drop(['country_region_code',
          'country_region',
          'sub_region_1',
          'sub_region_2',
          'metro_area',
          'iso_3166_2_code',
          'census_fips_code',
          'place_id'],
         axis='columns',
         inplace=True)
df2['date'] = pd.to_datetime(df2['date'])
df2.rename(columns={'retail_and_recreation_percent_change_
from_baseline': 'retail_and_recreation_change'},
inplace=True)

# Pick up the first 322 rows
df2 = df2.head(321)

print(df2.sample(10).to_string())
```

The only additional to the previous code is that we have renamed the `retail_and_recreation_percent_change_from_baseline` column to `retail_and_recreation_change` as it is shorter to reference. The output from the above code is:

```
          date  retail_and_recreation_change
grocery_and_pharmacy_percent_change_from_baseline
parks_percent_change_from_baseline
transit_stations_percent_change_from_baseline
workplaces_percent_change_from_baseline
residential_percent_change_from_baseline
183 2020-08-16                          -29.0
-18.0                                  52.0
-32.0                                      -10.0
3.0
287 2020-11-28                          -55.0
-12.0                                 -11.0
-55.0                                      -24.0
12.0
270 2020-11-11                          -49.0
-14.0                                  -1.0
```

```
-51.0                                        -38.0
16.0
121 2020-06-15                             -46.0
-14.0                                    63.0
-50.0                                        -50.0
18.0
230 2020-10-02                             -30.0
-10.0                                     4.0
-37.0                                        -32.0
11.0
262 2020-11-03                             -17.0
-6.0                                     6.0
-39.0                                        -33.0
11.0
97  2020-05-22                             -66.0
-18.0                                    14.0
-59.0                                        -58.0
24.0
42  2020-03-28                             -83.0
-39.0                                   -54.0
-74.0                                        -57.0
20.0
221 2020-09-23                             -27.0
-11.0                                    20.0
-40.0                                        -35.0
11.0
251 2020-10-23                             -28.0
-6.0                                    19.0
```

Merging two DataFrames

We now have two DataFrames both containing data where there are associated dates and COVID related information. As we would like to work with these two DataFrames together, we will merge the two DataFrames we have previously obtained, using the Pandas `merge()` function. This will join the two DataFrames based on the date with each row holding data from both DataFrames:

```
df3 = pd.merge(df1, df2, on='date')
print(df3.sample(10).to_string())
```

The DataFrame held in the variable `df3` now holds all the columns from the previous two DataFrames:

```
         date  hospitalCases  newAdmissions  newCasesByPublishDate
retail_and_recreation_change
grocery_and_pharmacy_percent_change_from_baseline
parks_percent_change_from_baseline
transit_stations_percent_change_from_baseline
workplaces_percent_change_from_baseline
residential_percent_change_from_baseline
249 2020-10-21         8132.0         1259.0                  26688
-31.0                                                 -11.0
-2.0                                                -44.0
-35.0                                            13.0
269 2020-11-10        14964.0         1848.0                  20412
-48.0                                                 -13.0
```

```
14.0                                          -49.0
-39.0                                     15.0
64  2020-04-19        19574.0         1657.0                    5850
-80.0                                                -40.0
-12.0                                          -66.0
-49.0                                     14.0
178 2020-08-11         1039.0          114.0                    1148
-13.0                                                -11.0
118.0                                          -41.0
-48.0                                     12.0
153 2020-07-17         1804.0          144.0                     687
-33.0                                                -10.0
93.0                                          -41.0
-44.0                                     13.0
301 2020-12-12        16909.0         1799.0                   21502
-37.0                                                 -5.0
-5.0                                          -39.0
-12.0                                      8.0
198 2020-08-31          854.0           91.0                    1406
3.0                                              -17.0
152.0                                          -43.0
-68.0                                     14.0
33  2020-03-19            NaN            NaN                     643
-21.0                                                 24.0
2.0                                           -37.0
-27.0                                     11.0
38  2020-03-24            NaN         1720.0                    1427
-71.0                                                -24.0
-14.0                                          -64.0
-58.0                                     24.0
183 2020-08-16          979.0           80.0                    1040
-29.0                                                -18.0
52.0                                          -32.0
-10.0                                      3.0
```

Analysing the Combined Data

We can now look at the relationships between data from the UK government and from Google COVID mobility portals. For example, we can look to see if there are any positive or negative correlations between this data:

```
print(df3.corr().to_string())
```

This produces the following grid:

```
                                                    hospitalCases
newAdmissions  newCasesByPublishDate  retail_and_recreation_change
grocery_and_pharmacy_percent_change_from_baseline
parks_percent_change_from_baseline
transit_stations_percent_change_from_baseline
workplaces_percent_change_from_baseline
residential_percent_change_from_baseline
hospitalCases                                           1.000000
0.900970              0.637143                     -0.551667
-0.251541                           -0.733562
-0.689504                                -0.359466
0.530903
newAdmissions                                           0.900970
1.000000              0.618497                     -0.490277
```

```
-0.278846                                -0.736805
-0.672790                                     -0.374371
0.521693
newCasesByPublishDate                                          0.637143
0.618497                1.000000                       -0.074673
0.081381                               -0.335174
-0.251770                                     -0.069931
0.127715
retail_and_recreation_change                                  -0.551667      -
0.490277               -0.074673                        1.000000
0.765417                                0.375138
0.890613                                      0.647121
-0.766577
grocery_and_pharmacy_percent_change_from_baseline             -0.251541      -
0.278846                0.081381                        0.765417
1.000000                                0.212263
0.698326                                      0.549779
-0.539766
parks_percent_change_from_baseline                            -0.733562      -
0.736805               -0.335174                        0.375138
0.212263                                1.000000
0.267764                                      0.036072
-0.249077
transit_stations_percent_change_from_baseline                 -0.689504      -
0.672790               -0.251770                        0.890613
0.698326                                0.267764
1.000000                                      0.839920
-0.882870
workplaces_percent_change_from_baseline                       -0.359466      -
0.374371               -0.069931                        0.647121
0.549779                                0.036072
0.839920                                      1.000000
-0.939791
residential_percent_change_from_baseline                       0.530903
0.521693                0.127715                       -0.766577
-0.539766                               -0.249077
-0.882870                                     -0.939791
1.000000
```

There is a lot of data here but one of the things we can see is that there appears to be a negative correlation between `HospitalCases` and `retail_and_recreation_change`.

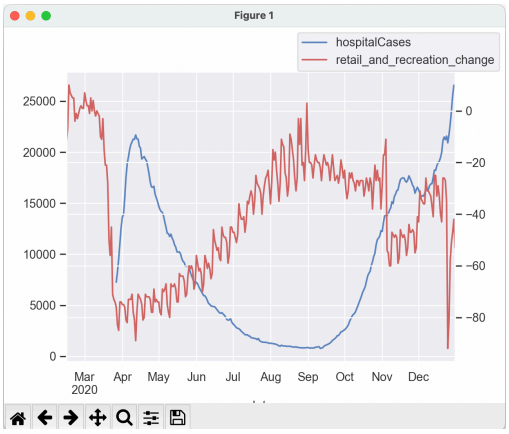
This might be useful to visualize on a graph. We can therefore generate a new DataFrame containing just the date and the `HospitalCases` and `retail_and_recreation_change` data:

```
df4 = pd.concat([df3['date'],
                 df3['hospitalCases'],
                 df3['retail_and_recreation_change']], axis=1)
```

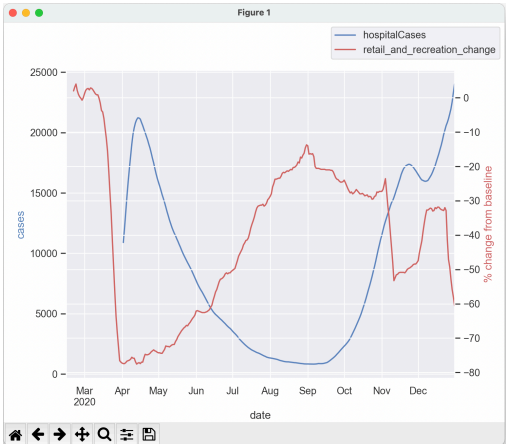
We can now choose to plot this data in different ways. For example, we might decide to generate a simple line graph that illustrates the relationship between these two data columns. To do this we are using Matplotlib and Seaborn, this means we need to import those libraries so that they are accessible to our program and initialise the graphing configuration values using `sns.set()`:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()

axis1 = df4.plot(x="date", y="hospitalCases", legend=False)
axis2 = axis1.twinx()
df4.plot(x="date", y="retail_and_recreation_change",
         ax=axis2, legend=False, color="r")
axis1.figure.legend()
plt.show()
```

Note that because the two columns have different value ranges, we need to plot them on two separate axes to see their respective changes. To do this we create two axis objects one for the `hospitalCases` and one for the `retail_and_recreation_change` data. We then plot the graph. The resulting graph is shown below:



This graph does highlight the trends in the data but there is certainly quiet a lot of noise in the red line. We might therefore decide to smooth out the date as bit to see the general trends. We can do this by smoothing out each data point by taking an average across the 7 preceding days. This can be done using the `rolling()` function and then taking the mean of the rolling values. These new values can then also be graphed and appropriate labels added:



In conclusion

We can quickly build, manipulate and visualise data thanks to Python and free open source tools such as Pandas, Matplotlib and Seaborn.

In this relatively simple instance we've brought together two sets of freely downloadable UK data to display how the number of COVID-related hospital cases correlated to the movement of shoppers in 2020 - the relationship between the two factors is rather striking and a good demonstration of the power of data visualisation.

You can apply these data analysis techniques to a huge variety of data types, at scale. It is possible to extract and refine valuable information from virtually any source.

Please note the Google Mobility Datasets are available only for a limited time.

GitHub Repo

All examples used in this blog along with the sample data are available in **https://github.com/johnehunt/python-covid-data-analysis/tree/main/pandas_data_analysis**.